



EXVUL

SMART CONTRACT **AUDIT REPORT**

Rolldex Bridge Smart Contract

MAY 2026

Contents

1. EXECUTIVE SUMMARY	4
1.1 Methodology	4
2. FINDINGS OVERVIEW	8
2.1 Project Info And Contract Address	8
2.2 Summary	8
2.3 Key Findings	9
2.4 Contract Files MD5 Checksums	10
3. DETAILED DESCRIPTION OF FINDINGS	11
3.1 Inbound Payload Settlement Proceeds Without Any Operator-Controlled Brake	11
3.2 Reliance on LayerZero Default Verification Path Concentrates Message Authority on a Single Operator	13
3.3 Peer Changes Can Invalidate In-Flight Messages	15
3.4 LayerZero Libraries Are Not Explicitly Set Per Pathway	17
4. CONCLUSION	19
5.1 Basic Coding Assessment	19
5.1.1 Apply Verification Control	19
5.1.2 Authorization Access Control	19
5.1.3 Forged Transfer Vulnerability	20
5.1.4 Transaction Rollback Attack	20
5.1.5 Transaction Block Stuffing Attack	20
5.1.6 Soft Fail Attack Assessment	21
5.1.7 Hard Fail Attack Assessment	21
5.1.8 Abnormal Memo Assessment	21
5.1.9 Abnormal Resource Consumption	22
5.1.10 Random Number Security	22
5.2 Advanced Code Scrutiny	22
5.2.1 Cryptography Security	22
5.2.2 Account Permission Control	23
5.2.3 Malicious Code Behavior	23
5.2.4 Sensitive Information Disclosure	23
5.2.5 System API	24
6. DISCLAIMER	25

7. REFERENCES	26
8. About Exvul Security	27

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **Rolldex Bridge** to review smart contract implementation for a LayerZero V2 OFT cross-chain bridge between Base and BSC. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve each issue and additional suggestions for improvement.

The outcome of the assessment outlined in Chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

All four findings documented in this engagement were fixed by the Rolldex team. Remediation details are recorded in Section 3 under each finding.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood:** represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact:** measures the technical loss and business damage of a successful attack.
- **Severity:** determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

		Informational	Low	Medium	High
Likelihood	High	INFO	MEDIUM	HIGH	CRITICAL
	Medium	INFO	LOW	MEDIUM	HIGH
	Low	INFO	LOW	LOW	MEDIUM
		IMPACT			

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs:** We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing:** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations:** We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
Rolldex Bridge	13/05/2026 - 15/05/2026	Solidity

Repository

Private repository (delivered by Rolldex team)

Commit Hash

Audit delivery snapshot (May 2026)

2.2 Summary

Severity	Found
CRITICAL	0
HIGH	0
MEDIUM	1
LOW	2
INFO	1



2.3 Key Findings

Severity	Findings Title	Status
MEDIUM	Inbound payload settlement proceeds without any operator-controlled brake	Fixed
LOW	Reliance on LayerZero default verification path concentrates message authority on a single operator	Fixed
LOW	Peer changes can invalidate in-flight messages	Fixed
INFO	LayerZero libraries are not explicitly set per pathway	Fixed

Table 2.3: Key Audit Findings

2.4 Contract Files MD5 Checksums

Contract File	MD5
BasicToken.sol	20244646536a993a53c50e3638a83f64
MyAdapter.sol	c270967f1e55360cd06dfd8bceeddf9e
MyOFT.sol	d744cf6b5e8097123013efd63759dbcf

Table 2.4: Contract Files MD5 Checksums

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Inbound Payload Settlement Proceeds Without Any Operator-Controlled Brake

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

contracts/MyAdapter.sol

contracts/MyOFT.sol

DESCRIPTION:

`MyOFTAdapter._credit` and `MyOFT._credit` are reached through `EndpointV2.lzReceive`, which is declared `external payable` and has no caller restriction. Any externally owned account can re-fire a payload after the receive library has written its hash into `inboundPayloadHash`. The adapter only gates the outbound path via `_debit`:

```
// contracts/MyAdapter.sol
function _debit(
    address _from,
    uint256 _amountLD,
    uint256 _minAmountLD,
    uint32 _dstEid
) internal virtual override returns (uint256 amountSentLD, uint256
amountReceivedLD) {
    if (!bridgingEnabled && !whitelist[_from]) revert BridgingDisabled();
    return super._debit(_from, _amountLD, _minAmountLD, _dstEid);
}
```

`_credit` has no pause hook, so once a payload is verified any external account can call `endpoint.lzReceive` to push it through:

```
// LayerZero EndpointV2.sol
function lzReceive(
    Origin calldata _origin,
    address _receiver,
    bytes32 _guid,
    bytes calldata _message,
```

```
    bytes calldata _extraData
) external payable {
    _clearPayload(_receiver, _origin.srcEid, _origin.sender,
        _origin.nonce, abi.encodePacked(_guid, _message));
    ILayerZeroReceiver(_receiver).lzReceive{ value: msg.value }(_origin,
        _guid, _message, msg.sender, _extraData);
    emit PacketDelivered(_origin, _receiver);
}
```

Toggling `bridgingEnabled = false` only blocks new sends out of Base. Payloads already written into `inboundPayloadHash` remain settleable by anyone, and the operator has no on-chain lever to intercept them.

IMPACT:

When an abnormal cross-chain payload reaches `inboundPayloadHash` (for example, DVN compromise or receive-library bug), the operator cannot stop or delay its settlement. Any external account can trigger `endpoint.lzReceive` to push it through `_credit`, reducing incident response to observation only.

RECOMMENDATIONS:

Add an inbound pause guard on `_credit` for both `MyOFTAdapter` and `MyOFT`, and expose LayerZero `nilify` and `burn` so the operator can cancel a malicious payload after pausing inbound settlement.

REMEDIATION:

Will only allow whitelisted addresses to bridge funds. `bridgingEnabled` will be set to `false` when required funds are bridged.

3.2 Reliance on LayerZero Default Verification Path Concentrates Message Authority on a Single Operator

SEVERITY:

LOW

STATUS:

Fixed

PATH:

deployment configuration

DESCRIPTION:

LayerZero documentation explicitly warns that a default configuration may be a dead config, misconfigured, or contain placeholder dead DVNs that must be replaced before production (see the LayerZero integration checklist).

Critical (Must Complete)

- **Peers set on all pathways (bidirectional)**
A $\leftrightarrow$ B and B $\leftrightarrow$ A peers configured and verified on every chain.
- **DVN configuration set on all pathways**
Required and optional DVNs explicitly configured per pathway.
- **Executor configuration set on all pathways**
Max message size, executor address, and related parameters configured.
- **Enforced options configured for gas/value**
enforcedOptions set so users pay enough gas for destination execution.
- **Mock and test functions removed**
No leftover debug or example functions in production deployments.
- **Ownership and delegate addresses verified**
OApp owner, delegate, and upgrade admins set to the correct addresses.

Inbound `_credit` accepts any payload that the default required DVN commits. A single compromised or coerced operator key can sign a forged packet and drain the locked `BasicToken` collateral held by the adapter, following the same failure class observed in the KelpDAO incident.

IMPACT:

Message verification authority is concentrated on a single DVN under the default path. Forged cross-chain messages could release or mint bridged assets without a second independent attestation layer.

RECOMMENDATIONS:

Lock send and receive libraries for each pathway and require at least two independent DVNs before mainnet launch. Document the chosen DVN set and library versions in deployment runbooks and verify them on-chain after deployment.

REMEDIATION:

Will use two independent DVNs.

3.3 Peer Changes Can Invalidate In-Flight Messages

SEVERITY:

LOW

STATUS:

Fixed

PATH:

contracts/MyOFT.sol

contracts/MyAdapter.sol

DESCRIPTION:

Rolldex bridge execution is split into two stages: the source chain commits value first, and the destination chain credits later. In `OFTCore._send()`, the source chain executes `_debit(...)` before `_lzSend(...)`:

```
function _send(
    SendParam calldata _sendParam,
    MessagingFee calldata _fee,
    address _refundAddress
) internal virtual returns (MessagingReceipt memory msgReceipt,
    OFTReceipt memory oftReceipt) {
    (uint256 amountSentLD, uint256 amountReceivedLD) = _debit(
        msg.sender,
        _sendParam.amountLD,
        _sendParam.minAmountLD,
        _sendParam.dstEid
    );

    (bytes memory message, bytes memory options) =
        _buildMsgAndOptions(_sendParam, amountReceivedLD);
    msgReceipt = _lzSend(_sendParam.dstEid, message, options, _fee,
        _refundAddress);
}
```

This creates an in-flight window between source-side commit and destination-side execution. On the destination chain, peer trust is checked in `OFTCore.isPeer()` against the current stored value:

```
function isPeer(uint32 _eid, bytes32 _peer) public view virtual override
    returns (bool) {
```

```
    return peers[_eid] == _peer;
}
```

If `peer` is changed while a message is in flight, the message is sent under the old peer assumption but validated under the new one. The destination chain may then reject the message as an invalid peer or fail to complete the intended credit flow.

IMPACT:

Once triggered, the source-side transfer has already been committed before destination-side peer validation occurs: Base to BSC locks `BasicToken` in the adapter, while BSC to Base burns `MyOFT`. If the destination chain rejects the in-flight message under the updated peer configuration, the corresponding mint or release will not complete, resulting in stuck user funds.

RECOMMENDATIONS:

Make peer configuration write-once in code. Allow `setPeer` only when `peers[_eid] == bytes32(0)` and reject overwrites thereafter so in-flight messages cannot be invalidated by later peer rotation.

REMEDIATION:

Peer will be fixed after deployment. No changing peers actions.

3.4 LayerZero Libraries Are Not Explicitly Set Per Pathway

SEVERITY:

INFO

STATUS:

Fixed

PATH:

deployment configuration

DESCRIPTION:

In the visible Rolldex deployment flow, there is no explicit configuration of the LayerZero send library or receive library. The message pathway appears to rely on protocol defaults rather than explicitly pinned per-pathway libraries.

Recommended (Best Practices)

- **Using latest LayerZero packages**
Contracts imported from the latest published packages, not copied source.
- **Libraries explicitly set (no reliance on defaults)**
Send/receive libraries set per pathway instead of using protocol defaults.
- **Message safety checks implemented**
One action per message or robust handling for bundled actions.
- `msg.value` checks in `lzReceive` / `lzCompose`
Encoded and validated to prevent underfunded execution or unexpected state.

IMPACT:

This is best characterized as a best-practice gap rather than a confirmed vulnerability. The current implementation does not follow the recommended LayerZero practice of explicitly pinning send and receive libraries per pathway, which reduces configuration transparency and increases the risk of misconfiguration during future migrations, upgrades, or multi-chain expansion.

RECOMMENDATIONS:

During deployment, explicitly set and verify send and receive libraries for each Base-BSC pathway. Record library addresses in deployment artifacts and confirm them on-chain before enabling production bridging.

REMEDIATION:

The deploy configuration will use fixed libraries.

4. CONCLUSION

In this audit, we thoroughly analyzed **Rolldex Bridge** smart contract implementation. The problems found are described and explained in detail in Section 3. All issues documented in this engagement were fixed by the Rolldex team during audit remediation. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage. # 5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

- [1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). <https://cwe.mitre.org/data/definitions/191.html>.
 - [2] MITRE. CWE-197: Numeric Truncation Error. <https://cwe.mitre.org/data/definitions/197.html>.
 - [3] MITRE. CWE-400: Uncontrolled Resource Consumption. <https://cwe.mitre.org/data/definitions/400.html>.
 - [4] MITRE. CWE-440: Expected Behavior Violation. <https://cwe.mitre.org/data/definitions/440.html>.
 - [5] MITRE. CWE-684: Protection Mechanism Failure. <https://cwe.mitre.org/data/definitions/693.html>.
 - [6] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
 - [7] MITRE. CWE CATEGORY: Behavioral Problems. <https://cwe.mitre.org/data/definitions/438.html>.
 - [8] MITRE. CWE CATEGORY: Numeric Errors. <https://cwe.mitre.org/data/definitions/189.html>.
 - [9] MITRE. CWE CATEGORY: Resource Management Errors. <https://cwe.mitre.org/data/definitions/399.html>.
 - [10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology
-

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

Contact

 **Website**
www.exvul.com

 **Twitter**
[@EXVULSEC](https://twitter.com/EXVULSEC)

 **Email**
contact@exvul.com

 **Github**
github.com/EXVUL-Sec